ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HospitalSanitizationTracker

Blockchain and Cryptocurrencies – Università di Bologna

Francesco Castaldi

Febbraio 2026

Indice

1 Introduzione **3**
1.1 Obiettivi . . . 3
1.2 Valore pratico e utilità nel contesto ospedaliero . . . 3

2 Concetti teorici e tecnologie utilizzate **5**
2.1 Ethereum e smart contract . . . 5
2.2 Linguaggio Solidity . . . 5
2.3 Hardhat e Ignition . . . 5
2.4 Ethers.js, MetaMask e DApp frontend . . . 6

3 Requisiti e casi d'uso **7**
3.1 Requisiti sintetici . . . 7
3.2 Esempi di scenari reali . . . 7

4 Architettura **8**
4.1 Vista di alto livello . . . 8
4.2 Ruoli e responsabilità . . . 8

5 Smart contract `SanitizationTracker` **8**
5.1 Modello dati . . . 8
5.2 Eventi e controllo accessi . . . 9
5.3 Operazioni principali ed esempi . . . 10

6 Ambiente di sviluppo e deploy **11**
6.1 Stack tecnologico . . . 11
6.2 Test . . . 12
6.3 Deploy su Sepolia . . . 12

7 Frontend DApp **13**
7.1 Struttura dei file . . . 13
7.2 Connessione al contratto . . . 13
7.3 Esempio di flusso completo . . . 13

8 Sicurezza, limiti e possibili estensioni **14**
8.1 Aspetti di sicurezza . . . 14
8.2 Limiti e sviluppi futuri . . . 14

9 Conclusioni **14**

1 Introduzione

L'obiettivo di *HospitalSanitizationTracker* è tracciare su blockchain le attività di sanificazione in ambito ospedaliero. Ogni intervento eseguito da un operatore autorizzato viene registrato come evento immutabile su Ethereum (rete Sepolia), consentendo audit successivi e verifica indipendente da parte di terzi (ad esempio direzione sanitaria o autorità di controllo).

Nel contesto reale, le sanificazioni sono spesso documentate tramite moduli cartacei o sistemi informatici centralizzati. In entrambi i casi è difficile garantire che i dati non vengano modificati a posteriori o che una certa registrazione corrisponda effettivamente all'azione compiuta da un operatore specifico. Usare la blockchain introduce un'infrastruttura condivisa e append–only, in cui ogni transazione è firmata crittograficamente e ha un timestamp verificabile.

Il progetto nasce nell'ambito del corso *Blockchain and Cryptocurrencies* e si ispira alla Proposal #7 *DLTs for Traceability in Supply Chain*, applicata al dominio sanitario. Rispetto a una supply chain tradizionale di prodotti, qui gli "asset" da tracciare sono le attività di sanificazione di specifiche aree fisiche (sale operatorie, stanze di degenza, magazzini di dispositivi medici).

1.1 Obiettivi

Gli obiettivi principali che hanno guidato l'implementazione sono:

- Progettare un modello dati semplice ma espressivo per rappresentare aree, operatori e singoli eventi di sanificazione.
- Garantire una traccia non modificabile delle sanificazioni, sfruttando la persistenza immutabile della blockchain.
- Collegare ogni evento all'identità (indirizzo Ethereum) dell'operatore che ha firmato la transazione, in modo da avere responsabilità chiare.
- Distinguere i ruoli di *admin* (deployer del contratto e gestore dell'anagrafica) e *operator* (personale operativo che registra gli interventi).
- Fornire una DApp web semplice per eseguire tutte le operazioni senza ricorrere a strumenti da sviluppatore (CLI, script o console).

1.2 Valore pratico e utilità nel contesto ospedaliero

Nel contesto ospedaliero reale, la tracciabilità delle sanificazioni ha un impatto diretto su sicurezza del paziente, gestione del rischio clinico e conformità normativa. Ogni sala operatoria o area critica deve poter dimostrare, in caso di audit interno o esterno, quando è stata sanificata, da chi e con quali modalità.

Un sistema tradizionale basato su fogli firma o software centralizzati espone alcuni limiti ricorrenti: i log possono essere modificati a posteriori, è difficile garantire che l'utente che registra l'operazione corrisponda effettivamente all'operatore che l'ha svolta, e spesso l'accesso ai dati è frammentato tra reparti diversi. La blockchain, in questo scenario, fornisce un registro condiviso, append–only e firmato crittograficamente, che rende molto più difficile manipolare le evidenze a posteriori.

HospitalSanitizationTracker non si propone di sostituire i sistemi informativi ospedalieri esistenti, ma di agire come livello di "audit trail" ad alta affidabilità: le informazioni essenziali (area, operatore, esito, timestamp) vengono ancorate on-chain, mentre eventuali dettagli sensibili rimangono nei sistemi interni dell'ospedale. In questo modo si ottiene un compromesso ragionevole tra trasparenza, verificabilità e rispetto della privacy.

Dal punto di vista operativo, la distinzione tra ruolo di admin e operatori attivi permette di modellare in modo fedele la realtà di un reparto: la direzione o l'IT interno gestisce l'anagrafica e i permessi, mentre il personale sanitario registra gli interventi con il proprio wallet. In prospettiva, un sistema di questo tipo potrebbe essere integrato in procedure di accreditamento, ispezioni o indicatori di qualità, fornendo una base oggettiva per dimostrare che le finestre di sanificazione sono state rispettate.

2 Concetti teorici e tecnologie utilizzate

2.1 Ethereum e smart contract

Ethereum è una piattaforma blockchain general purpose che permette di eseguire codice arbitrario (smart contract) su una rete distribuita di nodi. A differenza di una semplice criptovaluta, Ethereum espone una macchina virtuale (EVM) che consente di modellare logiche applicative complesse con garanzia di esecuzione deterministica e stato condiviso tra tutti i partecipanti.

Gli smart contract sono programmi che vivono on-chain e definiscono regole immutabili per la gestione dei dati: ogni transazione che li invoca modifica lo stato solo se rispetta le condizioni codificate. Nel contesto di questo progetto, lo smart contract `SanitizationTracker` rappresenta il "registro ufficiale" delle aree, degli operatori e degli eventi di sanificazione, sostituendo un database centralizzato con uno stato distribuito e verificabile.

2.2 Linguaggio Solidity

Solidity è il linguaggio di programmazione principale per lo sviluppo di smart contract sulla EVM. È tipizzato staticamente, ha una sintassi simile a quella di JavaScript e introduce costrutti specifici come `mapping`, `event`, `modifier` e il concetto di `msg.sender` per identificare l'indirizzo che ha firmato la transazione.

Nel progetto, Solidity 0.8.20 viene utilizzato per definire:

- le strutture dati (`Area`, `Operator`, `SanitizationEvent`);
- i mapping per l'accesso efficiente allo stato (`areas`, `operators`, `areaEvents`);
- i modifiers di sicurezza (`onlyAdmin`, `onlyActiveOperator`);
- gli eventi on-chain che permettono di tracciare operazioni rilevanti e di interfacciarsi facilmente con strumenti come Etherscan.

2.3 Hardhat e Ignition

Hardhat è un framework di sviluppo per Ethereum che fornisce un ambiente locale di esecuzione, compilazione, testing e deploy dei contratti. Consente di automatizzare la toolchain: compilazione di Solidity, esecuzione dei test JavaScript/TypeScript e interazione con reti pubbliche o di test tramite provider RPC. [page:1]

In questo progetto Hardhat 2.28.0 viene utilizzato per:

- compilare il contratto `SanitizationTracker.sol`;
- eseguire la suite di test `SanitizationTracker.test.js` (14/14 test passati);
- gestire il deploy su Ethereum Sepolia tramite modulo Ignition, che permette di definire in modo dichiarativo i passi di deploy e di ripeterli in ambienti diversi.

2.4 Ethers.js, MetaMask e DApp frontend

Ethers.js è una libreria JavaScript pensata per interagire con Ethereum da applicazioni web o Node.js. Fornisce astrazioni per provider, wallet e oggetti `Contract`, rendendo semplice chiamare funzioni on-chain direttamente dal browser. [page:1]

MetaMask è un wallet non-custodial che gira come estensione del browser e gestisce account, firma delle transazioni e iniezione dell'oggetto `window.ethereum`. Nel progetto, la DApp frontend utilizza Ethers.js per:

- connettersi a MetaMask e ottenere l'account corrente;
- creare un'istanza di `ethers.Contract` puntata all'indirizzo deployato su Sepolia;
- chiamare le funzioni di lettura (es. `getAreaEvents`) e di scrittura (es. `sanitize`) direttamente dall'interfaccia HTML. [page:1]

In questo modo il browser diventa il "client" dell'applicazione distribuita: nessun server centrale applicativo è necessario, poiché la logica risiede nello smart contract e l'autenticazione è basata sulle chiavi del wallet dell'utente.

3 Requisiti e casi d'uso

3.1 Requisiti sintetici

Per rendere più leggibile la lista dei requisiti, essi vengono riassunti nella Tabella 1.

ID	Descrizione
RF1	L'admin registra aree identificandole con ID numerico e nome; le aree possono essere attivate/disattivate.
RF2	L'admin registra operatori associando indirizzo Ethereum, nome e stato (attivo/non attivo).
RF3	Un operatore attivo registra una sanificazione indicando area, esito e note libere.
RF4	Chiunque può vedere lo stato di un'area e l'ultima sanificazione registrata.
RF5	Chiunque può consultare lo storico completo degli eventi per una singola area.
RNF1	I record non sono modificabili o cancellabili una volta scritti su blockchain.
RNF2	Il sistema è accessibile da browser tramite MetaMask, senza installare software aggiuntivo lato utente.
RNF3	La logica di autorizzazione (admin/operator) è codificata nello smart contract, non solo nel frontend.
RNF4	Il contratto è coperto da test automatici per ridurre il rischio di regressioni.

Tabella 1: Requisiti funzionali e non funzionali

3.2 Esempi di scenari reali

Esempio 1 – Sala operatoria 101 Un ospedale decide di monitorare in modo rigoroso la sanificazione della sala operatoria 101. L'admin del sistema apre la DApp, si collega con il proprio wallet MetaMask e registra una nuova area con ID `101` e nome "Sala operatoria 101". Ogni mattina, prima dell'inizio degli interventi, l'operatore responsabile effettua la sanificazione e registra un evento tramite la DApp selezionando l'area 101, impostando l'esito su "OK" e inserendo come note "Prodotto A, contatto 15 minuti". In caso di audit, lo storico degli eventi per l'area 101 mostra una sequenza quotidiana di sanificazioni con timestamp e indirizzo di chi le ha registrate.

Esempio 2 – Disattivazione operatore Supponiamo che un operatore lasci temporaneamente il reparto. L'admin può chiamare la funzione `setOperatorActive(operator,false)`. Da quel momento, ogni transazione `sanitize` firmata da quell'indirizzo verrà rifiutata dal contratto, perché il modifier `onlyActiveOperator` richiede che l'operatore esista e sia attivo. In questo modo si evita che un ex–dipendente possa continuare a registrare operazioni a proprio nome.

4 Architettura

4.1 Vista di alto livello

L'architettura complessiva è volutamente minimale ma copre l'intero flusso: definizione della logica su blockchain, interazione tramite wallet e presentazione via web. La Tabella 2 riassume i livelli principali.

Livello	**Componenti**
On-chain	Smart contract `SanitizationTracker` su Ethereum Sepolia.
Off-chain	DApp web (HTML/JS) con `ethers.js` per la connessione alla rete.
Wallet utente	MetaMask che gestisce account, firma transazioni e fornisce `window.ethereum`.

Tabella 2: Vista sintetica dell'architettura

Il flusso tipico è il seguente: l'utente interagisce con la DApp nel browser, che tramite MetaMask ottiene l'account attivo e inoltra transazioni o chiamate in lettura al contratto su Sepolia. Lo smart contract valida le richieste (controllo ruoli, esistenza area, ecc.) e aggiorna lo stato on-chain oppure restituisce i dati richiesti.

4.2 Ruoli e responsabilità

In Tabella 3 vengono confrontati i due ruoli principali.

	Admin	**Operator**
Identità	Deployer del contratto (indirizzo salvato in `admin`).	Indirizzo registrato tramite `registerOperator`.
Azioni	Registra/attiva/disattiva aree e operatori.	Registra eventi di sanificazione su aree esistenti e attive.
Restrizioni	Uno solo per contratto.	Deve essere attivo, altrimenti le funzioni sensibili falliscono.

Tabella 3: Ruoli previsti dal sistema

5 Smart contract `SanitizationTracker`

5.1 Modello dati

Il contratto definisce tre struct centrali, riportate in Listato 1. Esse rappresentano rispettivamente un'area, un operatore e un singolo evento di sanificazione.

Listing 1: Struct principali

```
struct Area {
    uint256 id;
```

```
    string name;
    bool active;
    bool exists;
}

struct Operator {
    address wallet;
    string name;
    bool active;
    bool exists;
}

struct SanitizationEvent {
    uint256 areaId;
    address operatorAddress;
    uint256 timestamp;
    string outcome;
    string notes;
}
```

Per un accesso efficiente, sono utilizzate tre mapping:

- `areas[id]` restituisce i dati di una certa area.
- `operators[wallet]` restituisce i dati di un dato operatore.
- `areaEvents[areaId]` restituisce l'array di eventi relativi ad un'area.

L'indirizzo dell'admin (deployer) viene salvato nel costruttore e reso pubblico tramite la variabile `admin`.

5.2 Eventi e controllo accessi

Gli eventi e i modifiers principali sono mostrati in Listato 2.

Listing 2: Eventi e modifiers

```
event AreaRegistered(uint256 indexed areaId, string name);
event OperatorRegistered(address indexed operatorAddress, string name);
event AreaSanitized(uint256 indexed areaId, address indexed
   operatorAddress, string outcome);

modifier onlyAdmin() {
    require(msg.sender == admin, "Only admin can perform this action");
    _;
}

modifier onlyActiveOperator() {
    require(operators[msg.sender].exists, "Operator not registered");
    require(operators[msg.sender].active, "Operator not active");
```

```
    _;
}
```

Questa impostazione garantisce che:

- soltanto l'admin possa modificare l'anagrafica di aree e operatori;
- soltanto operatori registrati e attivi possano eseguire `sanitize`;
- gli eventi emessi contengano l'ID area e l'indirizzo dell'operatore, facilitando la ricerca tramite index su Etherscan.

5.3 Operazioni principali ed esempi

Registrare un'area

```
function registerArea(uint256 _id, string memory _name) public onlyAdmin {
    require(!areas[_id].exists, "Area already exists");
    areas[_id] = Area(_id, _name, true, true);
    emit AreaRegistered(_id, _name);
}
```

Esempio concreto Admin chiama `registerArea(101, "Test 101")` da MetaMask. Dopo la conferma, una chiamata in lettura a `areas(101)` restituirà:

- `id = 101`
- `name = "Test 101"`
- `active = true`
- `exists = true`

Registrare un operatore

```
function registerOperator(address _wallet, string memory _name) public
   onlyAdmin {
    require(!operators[_wallet].exists, "Operator already exists");
    operators[_wallet] = Operator(_wallet, _name, true, true);
    emit OperatorRegistered(_wallet, _name);
}
```

Esempio Admin registra `0x3367...aa963` come operatore, con nome "Operatore Sepolia". Se in seguito questo wallet esegue `sanitize`, il modifier `onlyActiveOperator` lo riconoscerà come valido.

Registrare una sanificazione

```
function sanitize(
    uint256 _areaId,
    string memory _outcome,
    string memory _notes
)
    public
    onlyActiveOperator
{
    require(areas[_areaId].exists, "Area does not exist");
    require(areas[_areaId].active, "Area is not active");

    SanitizationEvent memory newEvent = SanitizationEvent({
        areaId: _areaId,
        operatorAddress: msg.sender,
        timestamp: block.timestamp,
        outcome: _outcome,
        notes: _notes
    });

    areaEvents[_areaId].push(newEvent);
    emit AreaSanitized(_areaId, msg.sender, _outcome);
}
```

Esempio Operatore registra `sanitize(101, "OK", "Prodotto X, 10 min")`. L'evento viene aggiunto all'array `areaEvents[101]`, con `operatorAddress` uguale a `msg.sender` e `timestamp` impostato al tempo corrente del blocco.

Lettura dello storico

```
function getAreaEvents(uint256 _areaId)
    public
    view
    returns (SanitizationEvent[] memory)
{
    return areaEvents[_areaId];
}
```

Dal punto di vista della DApp, questi valori vengono trasformati in JSON e mostrati nella sezione di interfaccia *Area Events*, permettendo all'utente di vedere la lista completa degli interventi.

6 Ambiente di sviluppo e deploy

6.1 Stack tecnologico

La Tabella 4 sintetizza gli strumenti utilizzati.

Componente	Versione / Scelta
Solidity	0.8.20
Hardhat	2.28.0
Node.js	v22
Rete	Ethereum Sepolia Testnet
Provider	Infura
Wallet	MetaMask

Tabella 4: Stack tecnologico

Hardhat viene utilizzato sia per compilare il contratto, sia per eseguire i test unitari, sia per gestire il deploy su Sepolia tramite script e modulo Ignition.

6.2 Test

La suite di test `SanitizationTracker.test.js` verifica tutti i casi principali:

- creazione di aree nuove e prevenzione di duplicati;
- registrazione di operatori e enforcement delle regole di accesso;
- esecuzione di `sanitize` da parte di operatori validi;
- fallimento di `sanitize` se l'area non esiste o è disattivata;
- corretto funzionamento delle funzioni di lettura (conteggio eventi, ultimo evento).

L'esecuzione di `npx hardhat test` restituisce 14/14 test passati, evidenziando la solidità del contratto nelle condizioni previste.

6.3 Deploy su Sepolia

Il contratto è stato deployato sulla rete *Ethereum Sepolia Testnet*. I dati principali sono:

- Indirizzo contratto: `0x679C6625f9479cf3b711F7a246C8F7a6655E4517`.
- Data deploy: 21 Febbraio 2026.
- Provider RPC: Infura, configurato tramite file `.env`.

La DApp utilizza questo indirizzo all'interno del file `app.js` (costante `CONTRACT_ADDRESS`) per istanziare l'oggetto `ethers.Contract`.

7 Frontend DApp

7.1 Struttura dei file

File	Ruolo
`index.html`	Definisce la struttura della pagina, divisa in sezioni per admin, operator e consultazione.
`app.js`	Contiene la logica applicativa: connessione a MetaMask, chiamate al contratto, aggiornamento UI.
`style.css`	Fornisce uno stile base per rendere leggibili i form e separare visivamente le sezioni.

Tabella 5: File principali del frontend

7.2 Connessione al contratto

La connessione a MetaMask e al contratto avviene tramite il seguente codice JavaScript:

```
const accounts = await window.ethereum.request({
  method: "eth_requestAccounts",
});
currentAccount = accounts[0];

provider = new ethers.BrowserProvider(window.ethereum);
signer   = await provider.getSigner();
contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);
```

Successivamente la DApp chiama `contract.admin()` e `contract.operators(currentAccount)` per determinare se l'utente corrente è admin, operatore o semplice guest. In base al ruolo, alcuni pulsanti vengono abilitati o disabilitati, riducendo il rischio di errori lato utente (anche se la sicurezza vera è nel contratto).

7.3 Esempio di flusso completo

Un flusso tipico di utilizzo è il seguente:

1. L'admin (Account 1 su MetaMask) apre la DApp, seleziona la rete Sepolia e clicca *Connect MetaMask*. La UI mostra *Connected: 0x...* e *Role: admin*.

2. L'admin registra l'area 101 "Test 101" e un operatore usando l'indirizzo dell'Account 2.

3. L'admin cambia account in MetaMask selezionando l'Account 2 (operatore) e clicca nuovamente *Connect MetaMask*; ora la UI mostra *Role: operator*.

4. L'operatore compila il form *Record Sanitization* con *Area ID = 101*, *Outcome = OK*, *Notes = Test* e invia la transazione.

5. Dopo la conferma, la sezione *Area Status* per ID 101 mostra l'ultima sanificazione con timestamp e indirizzo dell'operatore; la sezione *Area Events* mostra l'array completo degli eventi memorizzati per quell'area.

8 Sicurezza, limiti e possibili estensioni

8.1 Aspetti di sicurezza

La sicurezza logica è principalmente affidata allo smart contract:

- i modifiers `onlyAdmin` e `onlyActiveOperator` impediscono usi impropri delle funzioni di scrittura;
- tutte le transazioni sono firmate tramite MetaMask, quindi non è necessario gestire chiavi private nel codice;
- la natura append–only della blockchain garantisce che gli eventi registrati non possano essere cancellati o modificati.

8.2 Limiti e sviluppi futuri

Pur essendo completo per un progetto accademico, il sistema presenta margini di miglioramento:

- i dati memorizzati on-chain sono volutamente essenziali (niente dati sensibili del paziente o dettagli eccessivi); un sistema reale richiederebbe integrazioni con database off-chain, magari cifrati;
- non sono gestiti ruoli intermedi (ad esempio auditor o responsabili di reparto) che potrebbero avere privilegi di sola lettura avanzata;
- il frontend, scritto in HTML/JS vanilla, potrebbe essere migrato a React o ad un framework moderno per migliorare la manutenzione e l'estensibilità.

9 Conclusioni

HospitalSanitizationTracker mostra come un singolo smart contract, abbinato a una DApp leggera, sia sufficiente per costruire un sistema di tracciabilità credibile per le sanificazioni ospedaliere. La distinzione tra admin e operatori, unita alla persistenza immutabile on-chain, offre una base solida per dimostrare agli esaminatori la comprensione sia degli aspetti tecnici di Solidity e Hardhat, sia del valore applicativo delle DLT in un contesto reale.

In prospettiva, il progetto potrebbe essere esteso includendo analisi statistiche degli eventi, notifiche automatiche in caso di mancata sanificazione entro certe finestre temporali e integrazione con standard sanitari esistenti. Queste estensioni dimostrerebbero come, partendo da un contratto relativamente semplice, sia possibile costruire applicazioni di tracciabilità sempre più vicine alle esigenze del mondo reale.